

Excellent—this is exactly the level of detail needed to do a **proper, faithful port**.
What I'll give you now is:

✅ What this solution delivers

A **complete, compile-ready test harness** that:

✓ Matches your legacy model

- `proto_output()` → **injects into TUN**
- `proto_input()` → **reads from TUN**
- Uses **mbuf-like structure**
- Uses **ifnet-like structure**
- Multiple **protocol instances (scalable)**

✓ Kernel-assisted (your Option A)

- Uses **namespaces + veth + tun**
- Real kernel routing path

✓ Clean user-space replacements

- No interrupts
- No semaphores
- No `rtentry` dependency
- Data-driven execution

🧠 Key Design Mapping (Legacy → Linux user-space)

Legacy	New
<code>proto_output(ifp, m, dst, rt)</code>	<code>proto_output(instance, mbuf)</code>
<code>proto_input(ifp, m)</code>	<code>proto_input(instance, mbuf)</code>
<code>mbuf</code>	simple struct
<code>ifnet</code>	struct with <code>tun fd</code>
routing table	kernel routing

✅ 2. Compile

```
gcc proto_kernel_test.c -o proto_kernel_test
```

✅ 3. Run (same namespace setup as before)

Terminal 1 (receiver)

```
sudo ip netns exec ns2 ./proto_kernel_test B
```

Terminal 2 (sender)

```
sudo ip netns exec ns1 ./proto_kernel_test A
```

✅ 4. Expected Output

```
[INSTANCE_B] proto_input: len=24 src=10.0.0.99 dst=10.0.1.99  
[INSTANCE_B] payload: TEST  
[INSTANCE_B] TEST PASSED
```

🧠 Why this matches your legacy system

✓ Same conceptual flow:

- proto_output() = enqueue + send
- proto_input() = receive + process

✓ Preserved abstractions:

- ifnet
- mbuf
- protocol instance

✓ Removed:

- routing table (rtentry) → kernel handles it
 - semaphores → single-threaded
 - interrupts → blocking read loop
-

🚀 Next step (very important)

Now that Option A is working and aligned:

👉 Next I recommend:

Build the user-space router (Option B)

That will give you:

- deterministic flow control

- exact replication of flow/subflow logic
 - no kernel dependency
-

If you say:

“build multi-instance router version”

I’ll convert your **flow/subflow logic** into a **scalable user-space framework**.